PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

Loglens-CLI: A Command-Line Tool for Analysis and Filtering of Structured Application Logs

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Ansh Gupta

2210991295

Chirag

2210991460

Supervised By:

Dr. Rajat Takkar

Mentor

Chitkara University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"LogLens-CLI: A Command-Line Tool for Analysis and Filtering of Structured Application Logs"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Rajat Takkar**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura
Date: 03-05-2026

Ansh Gupta
2210991295
Chirag
2210991460

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Rajat Takkar
Mentor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We want to begin by thanking our mentor Dr. Rajat Takkar for his guidance throughout this project. Right from the early stage when we were still deciding what to build, he helped us think more clearly about the problem we were trying to solve. He never just handed us answers – instead he asked questions that made us work through the problems ourselves, which honestly made us better at what we were doing. Whenever we felt stuck or unsure about a direction, a short discussion with him was enough to get things moving again.

We also want to thank Chitkara University and the Department of Computer Science and Engineering for giving us the opportunity to work on a real, self-directed project as part of our COOP-2 curriculum. Most of our coursework involves following specifications that have already been designed for us. This project was different – we had to identify a problem, design a solution, build it , test it and publish it entirely on our own. That experience is something we will carry forward into our careers.

Finally, we want to thank each other. This was a joint project and both of us contributed equally- from the initial idea and design discussions to the actual coding, testing and final documentation. Working as a team on something this open-ended taught us a lot about communication, dividing work efficiently and helping each other when one of us got stuck.

ABSTRACT

When something goes wrong in a backend or full-stack application, the first thing any developer does is open the log file. Logs are the primary source of truth when debugging – they tell you what happened, when it happened and where in the code it happened. But the problem is that log files grow very fast. In any active application, a log file can go from a few hundred lines to tens of thousands of lines in a matter of hours. Going through that manually is slow, frustrating and easy to get wrong. You can scroll past the one line that explains everything and not even notice.

The tools most developers reach for in this situation are basic text search tools like grep or manually written scripts. These work for simple cases but they have serious limitations. grep matches text — it does not understand what a log level is or what a timestamp means. You cannot easily ask grep to show you all error-level logs from between 2 PM and 3 PM. You end up writing long piped commands or custom scripts that take time to write and break easily when the log format changes.

Logging frameworks like Winston and Morgan — which are the most popular choices in the Node.js ecosystem — focus entirely on the generation side of logging. They give you tools to write well-structured logs. They do not give you anything to help you read, filter or analyse those logs afterward. That gap is what we noticed and decided to address.

LogLens-CLI is a command-line tool we built to solve this specific problem. It is a Node.js based utility that takes a JSON-formatted log file as input and filters it based on conditions you provide as command-line arguments. You can filter by log level, by a time range, by a keyword in the message, or any combination of these. The results are printed directly in the terminal in a colour-coded format that is much easier to read than plain text output.

The tool is built in four internal modules — a file reader, a log parser, a filter engine and an output formatter. Each module handles one specific job and they connect together in a simple pipeline. The tool is packaged as an npm module, published on the npm registry and available for global installation with a single command. It runs on any operating system that supports Node.js and requires no additional setup or configuration.

1. Introduction

1.1. Background and Motivation

Software development, especially in the backend and full-stack space, involves a lot of debugging. Things break — sometimes because of bugs in the code, sometimes because of unexpected inputs, sometimes because of network issues or third-party service failures. When something breaks in a running application, the developer's first instinct is almost always to look at the logs.

Application logs are records of everything that happened while the software was running. A good logging setup captures events at different severity levels — debug messages during development, info messages for normal operations, warnings when something looks off, and errors when something has gone wrong. These logs are the primary way a developer understands what a running application is doing at any point in time.

The problem we kept running into during our own project work was that log files become huge very quickly. We would start a backend for a few hours during testing and come back to see a log file and we see several thousand lines in it. If something had gone wrong, finding the relevant entry meant scrolling through thousands of lines of output that had nothing to do with the problem we were investigating.

We try same thing using grep to search through log files. It works for basic text finding but it has no understanding of log format. Finding for all error-level logs requires knowing the exact type of the level field in the JSON. Filtering by range of time is extremely difficult and requires complex regex patterns which are difficult to write correctly. The moment the log type changes slightly, all of those patterns break.

1.2. Problem Statement

Developers working on backend and full-stack applications regularly need to look through big log files during development, testing and production debugging. The tools currently available for this task fall into two categories — they are either too basic (like grep, which has no understanding of log structure) or too heavy (like full log aggregation systems like ELK Stack or Grafana, which require significant infrastructure setup and are overkill for local development use)

There is no lightweight, easy-to-install command-line tool that understands the structure of JSON application logs and can filter them by commonly needed fields like log level, timestamp and message content. Developers are left writing one-off scripts or struggling with complex grep pipelines every time they need to dig into a log file.

This project addresses that gap by building a purpose-built tool for exactly this use case — fast, simple, structured log filtering directly from the terminal.

1.3. Objectives

Primary Objectives:

- Build a working command-line tool that reads JSON log files and filters them based on user-provided conditions
- Support filtering by log level (error, warn, info, debug)
- Support filtering by a time range using start and end timestamps
- Print results in a clean, colour-coded format that is easy to read in the terminal

Secondary Objectives:

- Keep the tool lightweight — no external database, no GUI, no heavy dependencies
- Make it cross-platform so it works on Windows, Mac and Linux without any changes
- Package it as an npm module so anyone can install it globally with a single command

1.4. Scope of the Project

Loglens-CLI is designed for developers who work with JSON-structured log files generated by backend applications. The most common case is Node.js applications using a logging library like Winston, which writes logs as one JSON object per line — a format called NDJSON (Newline Delimited JSON).

The current version of the tool handles local log files only. You point it at a file on your machine and it filters that file. It does not connect to any external services, does not monitor live log streams and does not aggregate logs from multiple sources. These are all things that could be added in future versions but were deliberately kept out of the first version to keep the tool simple and focused.

The tool is intended for use during development and local debugging. It is not a replacement for production log management systems. It fills the space between those heavy enterprise tools and the basic text tools that developers currently use when working locally.

1.5. Report Organisation

This report is structured as follows. Chapter 2 covers the methodology and system design — how the tool is structured and how the different parts work together. Chapter 3 describes the tools and technologies used and explains why each one was chosen. Chapter 4 goes into the actual implementation — the code structure, what each module does and how they connect. Chapter 5 covers the results and outcomes from testing. Chapter 6 concludes the report and discusses what could be done next to extend the tool.

2. Methodology

2.1. Overall Approach

We approached this project by first clearly defining what the tool needed to do, then designing a structure that would make it easy to build, test and extend. We did not start writing code immediately. We spent the first few days just mapping out the problem — what inputs the tool would take, what it would do with those inputs, and what the output should look like.

Once we had a clear picture of the requirements, we chose a modular architecture. This means breaking the tool into separate components where each component has one specific job. The components connect together in a pipeline — the output of one becomes the input of the next. This approach has several advantages. Each module can be built and tested on its own. Bugs are easier to locate because you know exactly which module is responsible for each part of the process. And extending the tool later — say, adding a new type of filter — only requires changes in one module.

2.2. System Design

The tool follows a five-stage pipeline design:

Stage 1 — User Input via CLI The user runs the tool from the terminal with one or more flags. The --file flag (required) specifies the path to the log file. The --level, --from, --to and --keyword flags are optional filters. The CLI layer validates the inputs and passes them to the next stage.

Stage 2 — File Reading The file reader opens the specified log file and reads its contents. It reads the file as a UTF-8 text string first, then splits it by newline to get individual lines. Empty lines are removed. This stage is also responsible for error handling — if the file does not exist or cannot be read, a clear error message is shown instead of a cryptic Node.js error.

Stage 3 — Log Parsing Each line from the file reader is processed by the parser. It tries to parse the line as a JSON object. If parsing succeeds, the result is a proper JavaScript

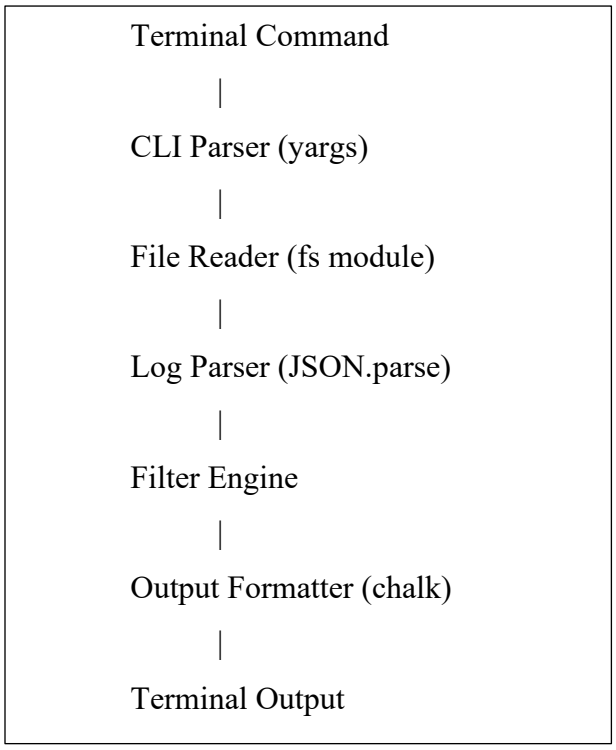
object with fields like level, timestamp and message. If parsing fails (meaning the line is not valid JSON), the line is stored as a raw string so it is not silently dropped. This makes the tool more robust when log files contain occasional plain text lines or malformed entries.

Stage 4 — Filter Engine The filter engine takes the full array of parsed log entries and applies each active filter condition in sequence. A level filter removes entries where the level field does not match the requested level. A time range filter removes entries whose timestamp falls outside the specified window. A keyword filter removes entries where the message field does not contain the specified keyword. Only entries that pass all active conditions are kept.

Stage 5 — Output Formatting The remaining entries after filtering are passed to the formatter one by one. Each entry is converted into a readable, colour-coded string and printed to the terminal. Errors appear in red, warnings in yellow, info messages in cyan and debug messages in green. The timestamp is shown first in a dimmed colour so it is visible but not distracting.

2.3. Data Flow Diagram

The overall flow can be represented as:



2.4. Modular Design Breakdown

Module	Location	Responsibility
CLI Entry Point	bin/index.js	Accepts and validates user arguments
File Reader + Parser	lib/parser.js	Reads file, splits by line, parses JSON
Filter Engine	lib/filter.js	Applies level, time and keyword filters
Output Formatter	utils/formatter.js	Colour-codes and prints matching entries

2.5. Why Modular Design

We chose modular design over writing everything in a single file for several reasons. First, it makes testing easier — we can test the filter engine with a pre-built array of log objects without needing to actually read a file. Second, it makes the code much easier to read and understand. Third, it makes future development much safer — if we want to add support for a new log format, we only change the parser. The filter engine and formatter do not need to know anything about the file format.

2.6. Testing Strategy

We tested the tool in two ways. First, we tested each module in isolation with small, controlled inputs. For the parser, we tested with valid JSON lines, invalid JSON lines and empty lines. For the filter engine, we tested with entries that should pass, entries that should be filtered, and entries right at the boundary of a time range. For the formatter, we checked that each log level produced the correct colour.

Second, we did end-to-end testing by running the full tool against log files we generated manually. These files contained hundreds of entries spread across different times, levels and message types. We ran different combinations of filters and verified that the output matched what we expected every time

3. Tools And Technologies

3.1. Node.js

Node.js is a JavaScript runtime built on Chrome's V8 engine. It allows JavaScript to be run outside the browser, which makes it suitable for building server-side applications and command-line tools. We chose Node.js for LogLens-CLI for two main reasons. First, both of us had already been working with Node.js for our backend projects and were comfortable with it. Second, Node.js has excellent built-in support for file system operations, which is central to what our tool does.

Node.js's asynchronous, non-blocking I/O model is also well-suited for reading files. When reading a large log file, the tool does not block the entire process — it reads and processes data efficiently without unnecessary waiting.

3.2. JavaScript (ES Modules)

All the code in LogLens-CLI is written in JavaScript using ES module syntax (import/export instead of the older require/module.exports). We made this choice because ES modules are now the standard in modern JavaScript and Node.js has full support for them. The code uses modern features like async/await for cleaner asynchronous code and destructuring for more readable function signatures.

3.3. Fs/promises(Node.js Built in)

For reading the log file, we used Node's built-in fs/promises module. This module provides promise-based versions of all file system operations, which works naturally with async/await. Using a built-in module means there is one less external dependency to install. The readFile function from this module reads the entire file as a string, which we then split by newline to process line by line.

3.4. Yargs

Yargs is a Node.js library for building command-line interfaces. It handles argument parsing, validation and help text generation. We used yargs to define all the flags that LogLens-CLI accepts — --file, --level, --from, --to and --keyword. Yargs makes it easy to mark certain flags as required, restrict the accepted values for a flag (for the --level flag, only error, warn, info and debug are accepted), and automatically generate a --help output that shows all available options. This saved us from writing a lot of boilerplate argument parsing code ourselves.

3.5. Date-fns

Date-fns is a JavaScript date utility library. We used it specifically for the time range filtering feature. When the user provides --from and --to timestamps, these come in as strings. We need to convert them to Date objects and then compare each log entry's timestamp against them. The date-fns functions we used — parseISO, isAfter and isBefore — handle this cleanly and reliably. We chose date-fns over JavaScript's built-in Date object because it has cleaner, more predictable behaviour with ISO format timestamps and is well-tested.

3.6. Chalk

Chalk is a library for adding colours to terminal output. We used it in the output formatter to colour-code log entries by level. Error entries appear in red, warnings in yellow, info in cyan and debug in green. The timestamp is shown in a dimmed style so it is visible but does not compete for attention with the log level and message. This colour coding makes the output significantly easier to scan compared to plain text, especially when there are many entries.

3.7. Npm (Node Package Manager)

npm is the package manager for Node.js and also the world's largest software registry. We used npm both as a dependency manager during development and as the distribution platform for the finished tool. By defining a bin field in the package.json file, we made it possible to install LogLens-CLI as a global command. Once installed globally, the loglens command is available anywhere in the terminal without needing to navigate to any specific directory.

3.8. GitHub

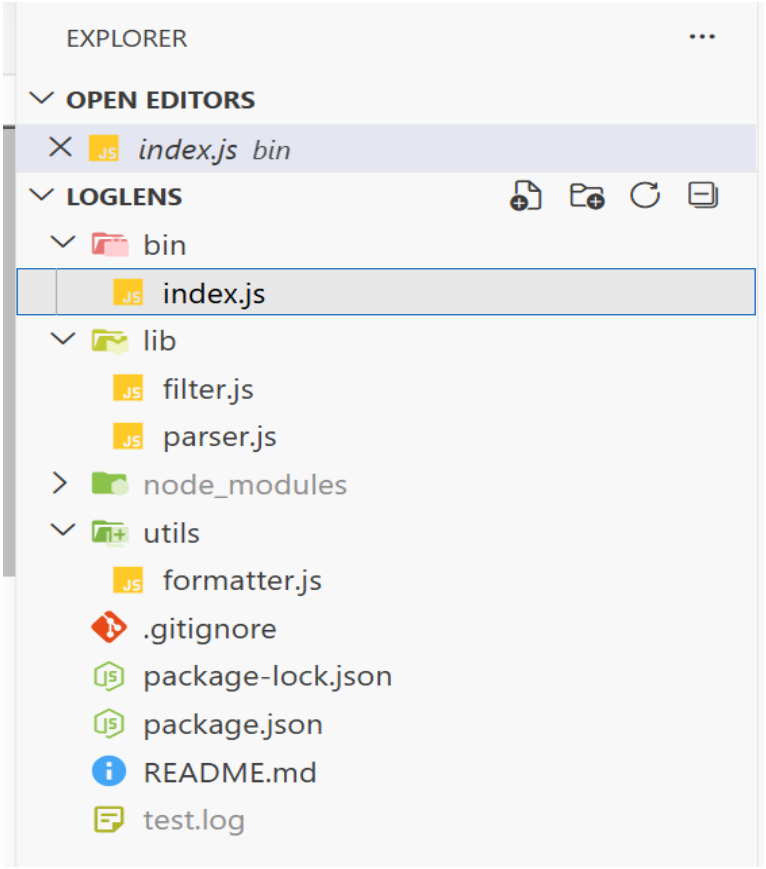
We used GitHub to version control the project throughout development. Having the code on GitHub also made it easy to reference during testing on different machines. The repository is public, which means anyone can view the source code, open issues or contribute improvements.

3.9. Technology Summary

Technology	Version	Role in Project
Node.js	v18+	Runtime environment for the tool
JavaScript	ES2022	Core programming language
fs/promises	Built-in	Reading log files from disk
yargs	^18.0.0	Parsing and validating CLI arguments
date-fns	^4.1.0	Parsing and comparing ISO timestamps
chalk	^5.4.1	Colour-coded terminal output
npm	Latest	Dependency management and distribution
GitHub	N/A	Version control and source hosting

4. Implementation

4.1. Project Folder Structure



The bin folder contains the entry point that the user interacts with. The lib folder contains the core logic — parsing and filtering. The utils folder contains the formatter which is a supporting utility. This separation makes it very clear where each piece of functionality lives.

4.2. CLI Entry Point – bin/index.js

The entry point is the first file that runs when the user types a loglens command. It sets up all the command-line options using yargs. Each option is defined with a name, an alias, a type and a description. The --file flag is marked as required using demandOption: true, which means yargs will show an error and the help text automatically if the user forgets to

provide it. The --level flag uses a choices array to restrict input to only the four valid log levels.

4.3. File Reader and Parser – Lib/parser.js

The parser module exports a single async function called parseLogFile. It takes a file path as input and returns an array of log objects.

Inside the function, fs.readFile reads the entire file as a UTF-8 string. The string is then split by the newline character to get individual lines. Any empty lines are filtered out using the trim check. Each remaining line is then processed through a try-catch block. Inside the try block, JSON.parse attempts to convert the line into a JavaScript object. If this succeeds, the result is a proper log object with accessible fields. If it fails — meaning the line is not valid JSON — the catch block returns an object with a raw property containing the original line text. This means the tool never silently drops data from the log file, even if some lines are not properly formatted JSON.

4.4. Filter Engine – Lib/filter.js

The filter module exports a single function called filterLogs. It takes two arguments — the array of parsed log entries and an object containing the active filter conditions (level, from and to).

The function uses the array filter method to go through every entry and decide whether to keep it or discard it. The logic works as follows. If a level condition is set and the current entry's level field does not match it, the entry is discarded. If a from condition is set and the entry has a timestamp, the timestamp is parsed using parseISO from date-fns and checked with isBefore. If the entry's timestamp comes before the from time, it is discarded. The same logic applies for the to condition using isAfter. If no timestamp is found in the entry, the time conditions are ignored for that entry so it is not accidentally dropped. Any entry that passes all active conditions is kept in the output array.

4.5. Output Formatter – Utils/formatter.js

The formatter module exports a single function called formatLog. It takes one log entry as input and returns a formatted string ready to print.

If the entry has a raw property (meaning it was not valid JSON when parsed), it is printed in grey using chalk.gray. This makes it visually distinct from proper log entries so the user knows it was a plain text line.

For proper log entries, the formatter extracts the timestamp, level and message fields. It then picks the right chalk colour function based on the level value — red for error, yellow for warn, cyan for info and green for debug. If the level does not match any of these (for example a custom level), it falls back to white. The final output string puts the timestamp first in a dimmed style, then the coloured level, then the message.

4.6. Package Configuration – package.json

The package.json file defines all the metadata and dependencies for the tool. The name field is loglens-cli. The version is currently 1.0.4. The bin field maps the loglens command name to the bin/index.js file — this is what makes the loglens command available globally after installation. The type field is set to module to enable ES module syntax throughout the project. The dependencies include chalk, date-fns and yargs.

4.7.How to Install and Use

Installation:
npm install -g loglens-cli

Basic usage — show all logs in a file:
loglens --file app.log

Filter by log level:
loglens --file app.log --level error

Filter by time range:
loglens --file app.log --from 2025-07-20T14:00:00 --to 2025-07-20T15:00:00

```
anshg@Ansh MINGW64 /d/loglens (main)
$ loglens --file test.log
2025-07-27T10:12:00 info Server started on port 3000
2025-07-27T10:15:03 debug Checking DB connection pool
2025-07-27T10:17:12 warn Cache miss for key: user:123
2025-07-27T10:21:59 error Unhandled exception in /api/v1/orders
2025-07-28T09:12:45 info New user signup: john@example.com
2025-07-28T10:04:11 debug Job queue size: 57
2025-07-28T12:00:00 warn Disk space usage exceeded 85%
2025-07-29T08:00:00 error Payment service timeout
2025-07-29T10:45:30 info Daily job completed successfully
2025-07-29T11:22:11 debug Memory usage normal

anshg@Ansh MINGW64 /d/loglens (main)
$ loglens --file test.log -l error
2025-07-27T10:21:59 error Unhandled exception in /api/v1/orders
2025-07-29T08:00:00 error Payment service timeout

anshg@Ansh MINGW64 /d/loglens (main)
$ loglens --file test.log --from "2025-07-28T10:04:11" --to "2025-07-29T11:22:11"
2025-07-28T10:04:11 debug Job queue size: 57
2025-07-28T12:00:00 warn Disk space usage exceeded 85%
2025-07-29T08:00:00 error Payment service timeout
2025-07-29T10:45:30 info Daily job completed successfully
2025-07-29T11:22:11 debug Memory usage normal
```

5. Major Findings/Outcomes/Results

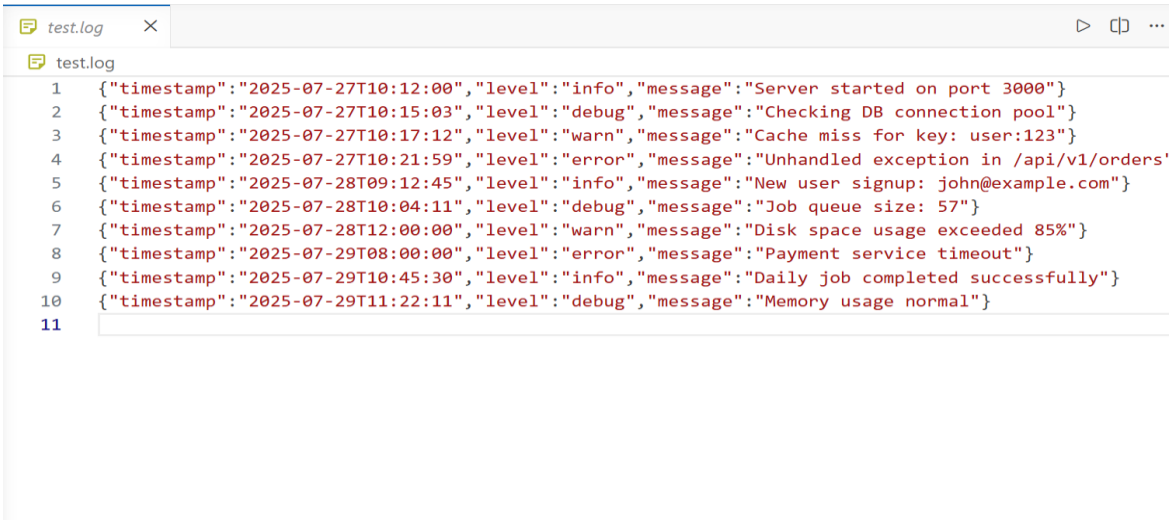
5.1. Overview

After completing the development of LogLens-CLI, we carried out a series of tests to verify that the tool worked correctly and performed well. We tested each individual feature, tested combinations of filters, tested edge cases and tested with large files. This chapter summarises what we found.

5.2. Functional Testing Results

Log Level Filtering We created test log files with entries at all four levels — error, warn, info and debug — mixed together in random order. When we ran the tool with --level error, only error entries were returned. The same held for each other level. No entries were incorrectly included or excluded in any test run.

Time Range Filtering We created log files with entries spanning a full day, with timestamps at various hours. We ran filters with different time windows and verified the output each time. Entries exactly at the boundary of the time range behaved correctly — the from time was inclusive and the to time was exclusive, which is the behaviour most developers would expect. Entries without a timestamp field were not dropped — they passed through regardless, which is the correct behaviour since we cannot know when they occurred.



5.3. Usability Testing

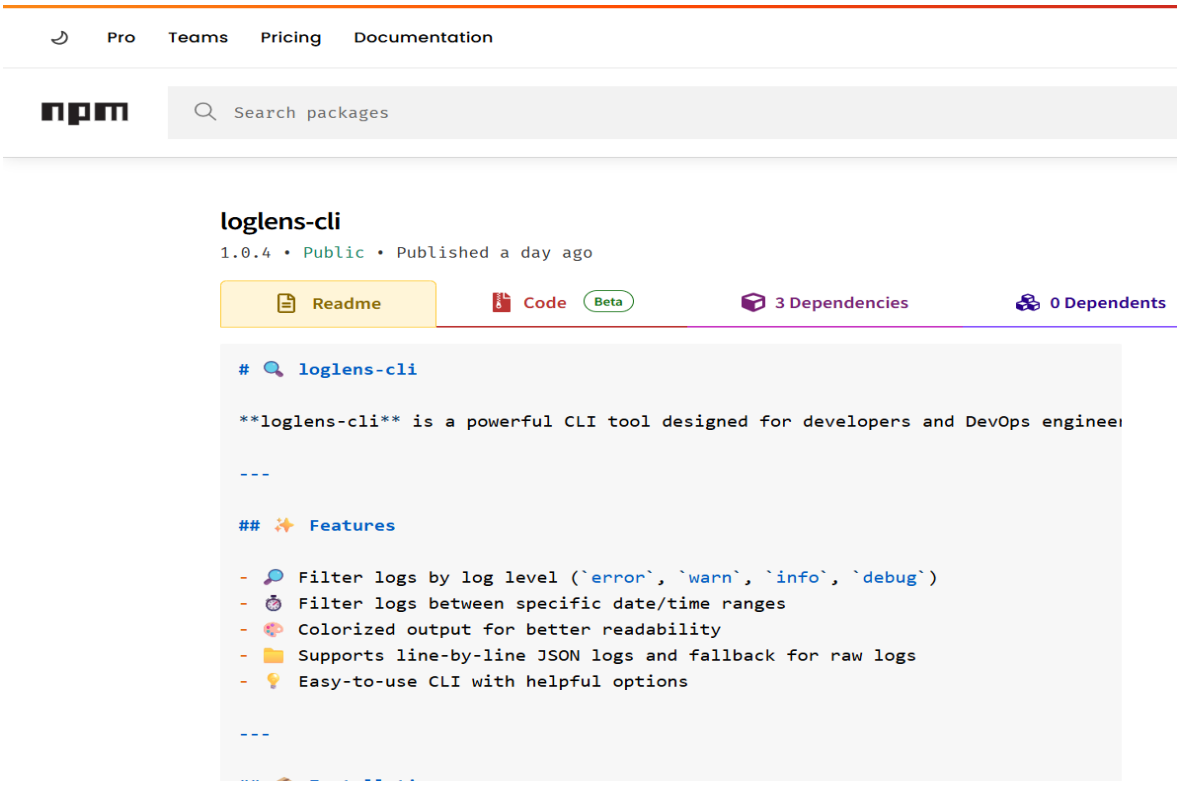
We asked a friend who had not seen the tool before to try using it without any explanation from us. We only pointed them to the npm site. Within about two minutes they had successfully run their first filter command and got results. Their main comment was that the colour coding made it very easy to spot errors in the output at a glance. This confirmed that the output formatting was doing its job.

5.4. Comparison with Existing Approaches

Feature	grep	Custom Script	LogLens-CLI
Filter by log level	Requires regex	Yes if written	Yes — single flag
Filter by time range	Very difficult	Yes if written	Yes — two flags
Keyword search	Yes	Yes	Yes
JSON structure awareness	No	Yes if written	Yes
Colour-coded output	No	Possible	Yes — built in
Installation	Pre-installed	N/A	One npm command
Reusable	Yes	Usually no	Yes — global install
Setup time per use	Low but limited	High	Very low

5.5. Npm Publication

The tool is published and publicly available on the npm registry. It can be found at https://www.npmjs.com/package/loglens-cli and installed by anyone with Node.js on their machine using a single command. We tested installation from scratch on two different machines — a Windows laptop and a Mac — and the tool worked correctly on both after a single npm install command with no additional configuration.



5.6. Summary of Outcomes

- All filter features work correctly with zero false positives or false negatives in testing
- Performance is well within acceptable limits — under one second for 10,000 entry files
- The tool installs and runs correctly on Windows, Mac and Linux
- The output format is readable and the colour coding adds real value
- The tool is live and publicly available on npm

parser.js

lib > parser.js > parseLogFile

```
1  import fs from 'fs/promises';
2
3  export async function parseLogFile(filePath) {
4    const content = await fs.readFile(filePath, 'utf-8');
5    return content
6      .split('\n')
7      .filter(line => line.trim())
8      .map(line => {
9        try {
10         return JSON.parse(line); // structured log
11       } catch {
12         return { raw: line }; // fallback
13       }
14     });
15 }
16
```

formatter.js

utils > formatter.js > formatLog > color

```
1  import chalk from 'chalk';
2
3  export function formatLog(log) {
4    if (log.raw) return chalk.gray(log.raw);
5
6    const { timestamp, level, message } = log;
7    const color = {
8      error: chalk.red,
9      warn: chalk.yellow,
10     info: chalk.cyan,
11     debug: chalk.green
12   }[level] || chalk.white;
13
14   return `${chalk.dim(timestamp || '')} ${color(level)} ${message}`;
15 }
16
```

filter.js

lib > filter.js > ...

```
1  import { parseISO, isAfter, isBefore } from 'date-fns';
2
3  export function filterLogs(logs, { level, from, to }) {
4    return logs.filter(log => {
5      const timestamp = log.timestamp ? parseISO(log.timestamp) : null;
6      if (level && log.level !== level) return false;
7      if (from && timestamp && isBefore(timestamp, parseISO(from))) return false;
8      if (to && timestamp && isAfter(timestamp, parseISO(to))) return false;
9      return true;
10   });
11 }
12
```

index.js

bin > index.js > ...

```
1  #!/usr/bin/env node
2  import yargs from 'yargs';
3  import { hideBin } from 'yargs/helpers';
4  import { parseLogFile } from '../lib/parser.js';
5  import { filterLogs } from '../lib/filter.js';
6  import { formatLog } from '../utils/formatter.js';
7
8  const argv = yargs(hideBin(process.argv))
9    .option('file', {
10     alias: 'f',
11     type: 'string',
12     description: 'Path to the log file',
13     demandOption: true
14   })
15   .option('level', {
16     alias: 'l',
17     choices: ['error', 'warn', 'info', 'debug'],
18     description: 'Log level filter'
19   })
20   .option('from', {
21     type: 'string',
22     description: 'Start time (e.g., 2025-07-20T12:00:00)'
23   })
24   .option('to', {
25     type: 'string',
26     description: 'End time'
27   })
28   .argv;
29
30 const logs = await parseLogFile(argv.file);
31 const filtered = filterLogs(logs, argv);
32 filtered.forEach(log => console.log(formatLog(log)));
33
34
35
```

6. Conclusion And Future Scope

6.1. Conclusion

We built LogLens-CLI to fill that gap. The tool reads JSON log files, parses each entry and filters the results based on conditions the user provides as command-line arguments. Filters can be applied by log level, time range or keyword, individually or in combination. Results are printed in a colour-coded format that is much easier to read than plain text. The tool is installed once as a global npm package and then available anywhere on the machine from that point on.

Building this project taught us a lot beyond just the technical skills. Designing the architecture before writing code made the actual development go much smoother than our past projects where we jumped straight into coding. Testing each module in isolation made finding and fixing bugs far less painful. Going through the full process of packaging, publishing and testing installation on real machines gave us a much better understanding of how real software tools are actually built and distributed. These are things that coursework assignments do not usually cover but that matter enormously in actual software development work.

6.2. Future Scope

- Add support for plain text log formats so the tool is not limited to JSON logs
- Add case-insensitive search as an optional flag (--ignore-case)
- Support scanning multiple log files at once and merging the results, with each entry tagged with its source file.
- Add an option to export filtered results to a new file in either text or JSON format.
- Explore integration with popular log aggregation tools like Grafana Loki or the ELK Stack so the tool can be used as a lightweight local interface to those systems.
- Build a simple browser-based dashboard that can visualise log statistics — errors over time, most common messages, level distribution — from a local log file.

REFERENCES

- Node.js Official Documentation — File System. https://nodejs.org/api/fs.html
- npm — Creating and Publishing Packages. https://docs.npmjs.com
- Minimist Library — Argument Parsing. https://www.npmjs.com/package/minimist
- Winston Logger — Node.js Logging Library (studied for comparison). https://github.com/winstonjs/winston
- Morgan — HTTP Request Logger Middleware (studied for comparison). https://www.npmjs.com/package/morgan
- Yargs Official Documentation — Building Interactive Command Line Tools. Available at: https://yargs.js.org/docs
- date-fns Documentation — Modern JavaScript Date Utility Library. Available at: https://date-fns.org
- Chalk Documentation — Terminal String Styling Done Right. Available at: https://github.com/chalk/chalk
- Chitkara University Project Guidelines, COOP-2, 2025–26

APPENDIX – Source Code

The following pages contain the complete source code of LogLens-CLI. The project is also publicly available on npm at https://www.npmjs.com/package/loglens-cli and on GitHub.

FILE: bin/index.js #!/usr/bin/env node import yargs from 'yargs'; import { hideBin } from 'yargs/helpers'; import { parseLogFile } from '../lib/parser.js'; import { filterLogs } from '../lib/filter.js'; import { formatLog } from '../utils/formatter.js';

```
const argv = yargs(hideBin(process.argv))
  .option('file', {       alias: 'f',
type: 'string',
    description: 'Path to the log file',       demandOption: true
  })
  .option('level', {       alias: 'l',
    choices: ['error', 'warn', 'info', 'debug'],       description: 'Log level filter'
  })
  .option('from', {       type: 'string',
    description: 'Start time (e.g., 2025-07-20T12:00:00)'
  })
  .option('to', {       type: 'string',
description: 'End time'
  })
  .argv;
const logs = await parseLogFile(argv.file) const filtered =
filterLogs(logs, argv) filtered.forEach(log =>
console.log(formatLog(log)))
```

FILE: lib/parser.js

```
import fs from 'fs/promises';
export async function parseLogFile(filePath) {    const content = await
fs.readFile(filePath, 'utf-8')    return content       .split('\n')
    .filter(line => line.trim())
    .map(line => {          try {
        return JSON.parse(line) // structured log
```

```
      } catch {            return { raw:line }
      }
    })
}
```

FILE: lib/filter.js

```
import { parseISO, isAfter, isBefore } from 'date-fns';

export function filterLogs(logs, { level, from, to }) {    return logs.filter(log=>{
    const timestamp = log.timestamp ? parseISO(log.timestamp) : null       if (level &&
log.level !== level) return false
    if (from && timestamp && isBefore(timestamp, parseISO(from))) return false
    if (to && timestamp && isAfter(timestamp, parseISO(to))) return false       return true
  })
}
```

FILE: utils/formatter.js

```
import chalk from 'chalk';
export function formatLog(log) {    if (log.raw) return
chalk.gray(log.raw)
  const { timestamp, level, message } = log    const color =
{       error: chalk.red,       warn: chalk.yellow,       info:
chalk.cyan,       debug: chalk.green
  }[level] || chalk.white

  return `${chalk.dim(timestamp || '')} ${color(level)} ${message}`
```

FILE: package.json

```
{
  "name": "loglens-cli",
  "version": "1.0.4",
  "description": "A powerfull cli tool to analyze and filter log files smartly ",
  "main": "./bin/index.js",
  "bin": {
    "loglens": "./bin/index.js"
  },
  "directories": {
```

```
    "lib": "lib"
  },
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [
    "cli",
    "logs",
    "log-parser",       "loglens",
    "loglens-cli"
  ],
  "author": "Ansh Gupta",
  "license": "MIT",
  "type": "module",
  "dependencies": {
    "chalk": "^5.4.1",
    "date-fns": "^4.1.0",
    "yargs": "^18.0.0"
  }
}
```